

A Computationally Inexpensive Generative Design Model for Airfoil and Vortex Generator Topology Optimization Through Data Augmentation

Grace Yu

Meadowridge School

April. 11, 2021

Abstract

Despite the rapid increase in computer processor speed, Computational Fluid Dynamics (CFD) analysis on complicated geometries remain inaccessible and may only be solved at supercomputer centers, expending hundreds of hours and astronomical cost while potentially yielding inefficient results [4]. To address the issue, we propose a computationally inexpensive machine learning (ML) model that combines the core concepts of CFD and generative design, to conduct optimization for airfoils and vortex generators (VG). Utilizing existing datasets from Baldacchino et al, XFLR5 simulation tool, and Python ML libraries, we created a model with an augmented reference dataset to guide both the search algorithms and the augmentation model to find and select optimal configurations, augment additional clusters of iterations surrounding some optimal designs, and further search for the most efficient airfoil and VG combination. After tuning the hyperparameters, the final data augmentation model may conduct batch analysis at the average speed of $5.57\text{e-}3$ ms per row, with a mean squared error (MSE) of 0.0078. It was also able to accurately predict unseen data, with MSE values ranging between 0.01 to $5.00\text{e-}5$. Depending on the breadth of the constraints, the final model was able to output n optimal designs within 50-80 seconds, significantly faster than CFD or other generative design models. However, the focus on cost reduction and usability hinders the flexibility of this program, reducing its potential usages. This algorithm is a step forward to the creation and prevalence of cost-efficient topology optimization models.

0 Glossary

Airfoil - a cross-sectional shape or a body that reacts in accordance with the lift theorem and may generate a significant amount of lift. Airfoils are often considered as one of the simplest aerodynamic shapes, commonly used in many industries, as wings, sails, turbines, rotors, etc.

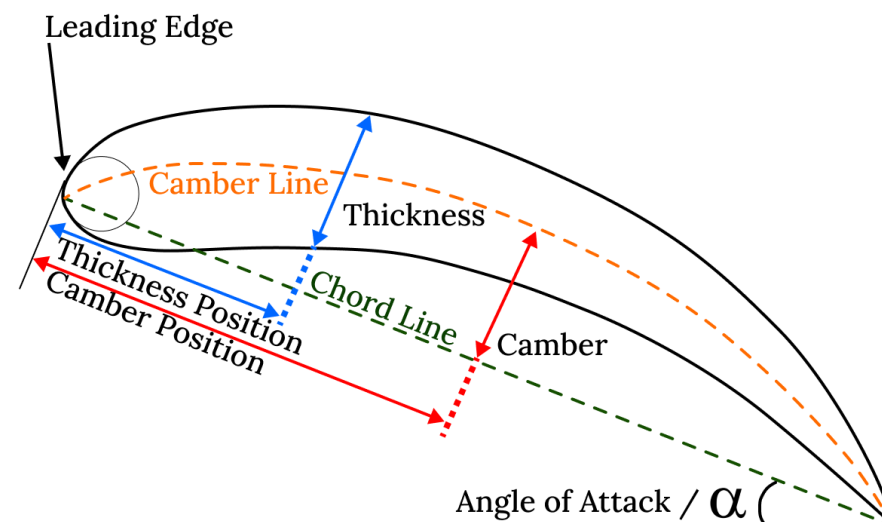


Figure 1: Airfoil geometry variables used in this experiment.

Camber - the measure of asymmetry or curvature of the airfoil. This is often recorded at its maximum point.

Camber Position - the distance between the point that has the greatest camber and the leading edge.

Thickness - the distance between the upper surface and the lower surface of the airfoil. This is often recorded at its maximum point.

Thickness Position - the distance between the point that has the greatest thickness and the leading edge.

Camber, camber position, thickness, and thickness position are measured in percentage in reference to the length of the chord line of the airfoil.

Vortex Generator (VG) - a boundary flow regulator that transforms laminar flow into vortices which may better follow the contours of aerodynamic bodies to improve the turbulence by better controlling separation flow, reducing the magnitude of the Coanda effect, and thus reducing drag.

Coanda Effect - the tendency of a fluid to stay attached to a convex surface. At high angle of attack, this effect is much weaker, thus leading to separation flow, creating turbulent wake.

VG Chord Position - A reference line normal to the leading edge of the airfoil is extended until it reaches the leading edge of the VG. The length of this line represents the chord position of the VG, measured in percentage in reference to the length of the airfoil.

VG Height - A reference line collinear to the leading edge of the VG is extended until it reaches the upper edge of the VG. The length of this line represents the height of the VG, measured in millimeters.

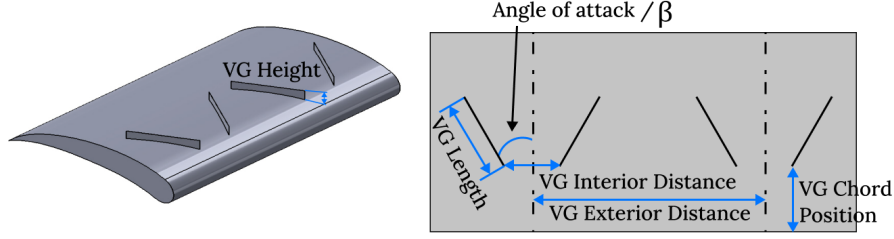


Figure 2: VG geometry variables used in this experiment.

VG Length - A linear reference line starting from the leading edge of the VG is extended until it reaches the trailing edge of the VG. The length of this line represents the length of the VG, measured in millimeters.

VG Interior Distance - VGs are often used in pairs. The distance between the leading edge of a pair of VG is its interior distance, measured in millimeters.

VG Exterior Distance - VGs are often used in pairs. The distance between the centerline of two distinct pairs of VGs is their exterior distance, measured in millimeters.

0.1 Notations

Aerodynamic Notations

α - angle of attack of the airfoil

β - angle of attack of the VG

cl - coefficient of lift

cd - coefficient of drag

cl/cd - lift to drag ratio

Variables in dataset

vg = VG

$boundary_1$ = Boundary Layer

g_chord_pos = VG Chord Position

vg_l = VG Length

vg_h = VG Height

vg_d_int = VG Interior Distance

vg_d_ext = VG Exterior Distance

vg_beta = VG Angle of Attack

$camber$ = Camber

$camber_pos$ = Camber Position

$thickness$ = Thickness

$thicknes_pos$ = Thickness Position

$alpha$ = Airfoil Angle of attack

Scientific Notations Example

$10000 = 1.00e4$

Other Notations 2D = two-dimensional 3D = three-dimensional

1 Introduction

1.1 Background Information

In recent years, the rapid advancement of Computational Fluid Dynamics (CFD) has had a staggering impact on many industries and engineering disciplines. After years of research and development, CFD is maturing into a versatile, pervasive tool that can accurately simulate the behavior of fluid flow [1]. However, despite the substantial improvement made in processor performance, iterative CFD is computationally expensive and relatively inaccessible.

Due to the complexity of the governing equations, the high dimensionality of the design space, and the high time resolution needed to evaluate turbulence means it may require hundreds of core hours to simulate the performance of one geometry iteration [2, 3]. High computational cost CFD problems typically are solved at national supercomputer centers or similar facilities that have limited access for academia and organizations outside of the government [4]. Due to the lack of access, engineers must make decisions with very little data or insights, potentially resulting in inefficient end products. Aerodynamic inefficiency can cause various deficits, such as increasing energy consumption or decreasing fuel economy, stability, and structural integrity. In a world that is rapidly progressing towards sustainability development, aerodynamic-driven topology optimization is a prominent field that may potentially elongate the lifespan of engineered systems by decreasing energy or fuel consumption.

Airfoils have many parameters that can be altered for increased efficiency. Fully simulating the behavior of airfoils in 3D may require high performance hardware and costly simulations. Conducting large quantities of these simulations may be unaffordable for most organizations outside of large corporations, high-level academia, or governments.

Machine learning (ML) is a relatively new concept which utilizes the basics of computational logic to extract insight from datasets and predict values. Using ML, computers can generate insight on a baseline dataset and augment large sums of data. Most common techniques of data augmentation involve using regressors, classifiers, or clustering to generate insights and predictions. In comparison to CFD, when producing data of similar quantity and quality, the computational cost of data generation using augmentation techniques is significantly lower [4].

When optimizing, engineers often take one of the following approaches: trial and error, grid search, or random search. For aerodynamicists, due to the cost of CFD, grid and random search are mostly unfeasible. Often, engineers only have the resources to simulate for a few potential solutions. As industries continue to conduct optimization research, a new concept called generative design has emerged. Generative design will use its backend dataset to generate insights, use user input to constrain its outputs and produce a few optimized potential design solutions. Generative Adversarial Networks (GANs), complex neural network systems that can produce outputs with high dimensionality and accuracy, are often used in generative design models [3].

1.2 Literature Review

In the past, many researchers have investigated the drag reduction effects of vortex generators (VG) on airfoils. Baldacchino et al discussed both the positive and negative effects of VG installation on thick airfoils in wind turbine applications, suggesting that VGs may delay dynamic stall. This will allow the airfoil to have a higher angle of attack, which may supply a higher peak lift, without experiencing an eminent increase in drag [6]. De Tavernier et al further corroborated with this claim by discussing how optimized VG configurations on airfoils may suppress dynamic stall in laminar airflow and maintain stability in turbulent airflow, although they may suffer from a drastic decrease in performance once a threshold is reached [7]. Forster and White also made investigations on installations of VGs in heavily cambered airfoils in similar conditions. Their findings revealed that heavily cambered airfoils are affected by VGs similarly to its thicker, less cambered counterparts [8]. This provides evidence that VGs may enhance airfoil performance under certain conditions.

In addition to drag reduction and optimization research on airfoils, researchers have also scrutinized the possibility of implementing ML to either reduce the computational costs or increase the accuracy of CFD. In 2000, Hauser et al investigated and developed a cluster model called the Kentucky Linux Athlon Testbed 2 (KLAT2) to lower the computational cost of direct numerical simulation of the Navier-Stokes equations (DNS), which is a commonly used CFD technique. They implemented this cluster model and achieved a much lower cost per million floating-points operations per second (MFLOPS). With single precision, the cost of simulation with I/O is 1.96 per MFLOPS; with double precision, the cost of simulation is 2.85 per MFLOPS [4]. In 2020, Zhao et al studies the possibility of using ML to increase the predictive accuracy of Reynolds-averaged Navier Stokes (RANS). They used gene expression programming (GEP), an evolution algorithm to actively train and improve the RANS model of visualization [9]. Jeon and Kim recently investigated the potential of implementing a finite volume method network (FVMN) to reduce the computational cost of CFD. Finite volume method (FVM), commonly used as in the foundational architecture of CFD software, is transitioned into a deep neural network and tuned to conduct similar tasks of traditional CFD. They found that FVMN with optimized hyperparameters performed significantly better than general networks, achieved a higher efficiency in multi-step predictions without compromising accuracy, and can perform various simulations more cost-effectively than traditional CFD yet with room for improvement [2].

In recent years, the rapid advancement of computational technology and the emergence of new design concepts prompted many researchers to study the implementation of generative ML models in aerodynamic-driven topology optimization. Chen et al investigated the implementation of ML techniques including GANs to synthesize optimized airfoils and decrease the computational cost of obtaining an optimal design [3]. Gunpinar et al explored the possibility of utilizing GAN architecture to predict the cd of sedans from its silhouette where they achieved a relatively high accuracy with much lower computational cost and processing time than simulating it via traditional CFD [10]. Bhatnagar et al attempted to construct a convolutional neural network (CNN) to predict the pressure and velocity of an unseen object given its pixelated cross-sectional geometry where they achieved higher efficiency, shorter simulation time than

the RANS solver with great accuracy[11].

1.3 Purpose and Proposed Solution

Topology optimization of airfoils and using ML to increase the performance of CFD are two very well-researched fields. However, there yet to be a bridge between them. CFD, generative design, and optimization software remain mostly separated while industries have developed very few products featuring merged functions from the aforementioned tools.

Each of these programs require meticulous geometry preparation procedures that are largely similar and each expends an immense amount of computational power. To reduce the computational cost, we propose a generative design model that utilizes existing data, data augmentation, clustering, and search algorithms to output optimized design options. Existing available datasets will be the initial input, branching into two tangent algorithms. Using the input dataset, we propose to train and optimize a data augmentation model to accurately generate additional insights. The real and augmented data will be combined and fed into the search algorithm to generate designs by creating clusters of potential geometrical combinations around pre-selected optimized configurations, augmenting their aerodynamic data, and find the n most optimized designs.

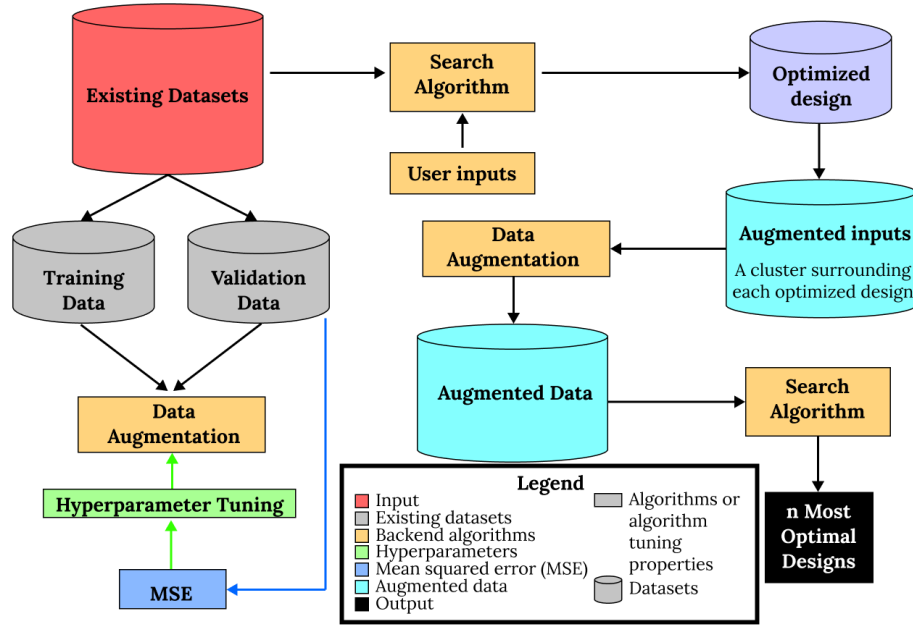


Figure 3: A rough outline of the proposed solution.

2 Methodology

This project may be classified into four major steps: data collection, data visualization, data augmentation, and final model construction.

2.1 Data Collection

Data used in this research are either from previous literature or computational simulations. An array of sixteen different airfoils are selected from the NACA, Gottingen, and Eppler airfoil series, forming a collection of airfoils that may numerically represent the aerodynamic properties of high lift, high speed, and general purpose airfoils. Airfoil geometries are exported from the Technical University of Delft Airfoil Coordinate Database, converted to comma separated values (CSV) files, and imported into XFLR5 for analysis. Note these analyses are conducted on airfoils with clean boundary conditions and without VG installations. Aerodynamic data will be exported and stored in a CSV file.

2.2 Materials

In addition to the collected data, the dataset from Baldacchino et al was used as part of the baseline dataset. During both data collection and data augmentation, XFLR5 was used to simulate aerodynamic data of airfoils. XFLR5 utilizes the lifting-line theory to simulate the aerodynamic properties of 2D cross-sectional shapes. Please refer to [12] for further explanations. We also adopted many Python libraries such as Numpy and Pandas to manipulate data. During data augmentation, SciKit Learn Python libraries were used due to the ease of accessibility and the low cost of their prebuilt decision tree algorithms.

2.3 Data Visualization

Both the collected data and the dataset from Baldacchino et al are combined, reformatted, and recompiled into the input dataset. To gain further insight into the dataset, we visualized portions of the data.

2.4 Data Augmentation

Through visualization, we recognized the non-linearity in the data. The exact correlation between α to cl , cd , and cl/cd are hard to identify, although they show semblance to transformed high degree polynomial correlations. Therefore, multiple linear regression or transformed regression may be unsuitable. We initially built and optimized a multilayer perceptron regression model. After extensive tuning and testing, the model was unable to accurately or consistently predict data with both original and normalized datasets. Therefore, due to its superior accuracy and low computational cost, ExtraTrees was chosen over the multilayer perceptron model (please refer to results for further justification).

During the data augmentation process, the baseline dataset is scrambled and split into two portions. Roughly 80% of the data became training data for the model, while the remaining are retained for validation.

After training and validating the model, a mean squared error (MSE) value will be calculated for accuracy evaluation. Once this value is minimized, a dataset consisting of an array of augmented geometric configurations of the sixteen selected airfoils will be input into the model, predicting for its aerodynamic data. In the end, these data will be exported into a CSV file, recombined with data from Baldacchino et al and XFLR5 simulations, and stored for further analysis.

Figure 4: Training and validation dataset summary.				
Datasets		Count	Mean	σ
Training	<i>vg</i>	900	0.675556	0.468427
	boundary_l	900	0.675556	0.468427
	vg_chord_pos	900	0.468427	0.992616
	vg_l	900	-0.015644	0.993989
	vg_h	900	-0.014943	0.995100
	vg_d_int	900	-0.014943	0.995100
	vg_d_ext	900	-0.010991	0.996500
	vg_beta	900	-0.000404	0.998505
	camber	900	1.0	0.0
	camber_pos	900	-1.0	0.0
	thickness	900	-5.018298e+01	1.279688e-13
	thinknes_pos	900	-5.018299e+01	1.777345e-13
	alpha	900	-0.006341	1.012752
	cl	900	-0.022016	1.011019
	cd	900	0.004143	1.003234
	cl/cd	900	-0.016836	1.003851
Validation	<i>vg</i>	276	0.942029	0.234113
	boundary_l	276	0.670290	0.470962
	vg_chord_pos	276	0.014530	1.025655
	vg_h	276	0.048400	1.016141
	vg_l	276	0.050742	1.019535
	vg_d_int	276	0.048400	1.016141
	vg_d_ext	276	0.035459	1.012357
	vg_beta	276	0.010712	1.018696
	camber	276	1.0	0.0
	camber_pos	276	-1.0	0.0
	thickness	276	-5.018298e+01	2.064317e-13
	thinknes_pos	276	-5.018299e+01	1.566034e-13
	alpha	276	0.023681	0.959967
	cl	276	0.075821	0.963503
	cd	276	-0.011700	0.991550
	cl/cd	276	0.052754	0.987909

Figure 4: Training and validation dataset summary.

2.5 Final Model

Search algorithms can easily be established by data manipulation, with low computational costs. However, its limitations include its lack of flexibility and its vulnerability to being hindered by inconclusiveness from baseline datasets. Therefore, to expand the scope of the search algorithm and improve its ability to locate optimized configurations, we propose to create a clustering mechanism that may augment for more optimal solutions using the data augmentation model.

The model initializes with the input dataset, and simultaneously branches into two distinct tangent algorithms. The first focuses on creating and tuning the data augmentation model by repeating the steps outlined in section 2.4,

the second focuses on producing the final outcome utilizing the aforementioned search algorithm and clustering mechanism.

The search algorithm utilizes the basic framework of data manipulation and boolean conditions to reconstruct the user inputs into constraints for filtering and sorting for the most optimal geometrical configurations. Then, it feeds the optimized designs into the clustering mechanism where it artificially generates a matrix of geometrical configurations that are similar to the optimized design. The matrix will be feed into the augmentation model and the search algorithm as input to further predict and filter for the best design.

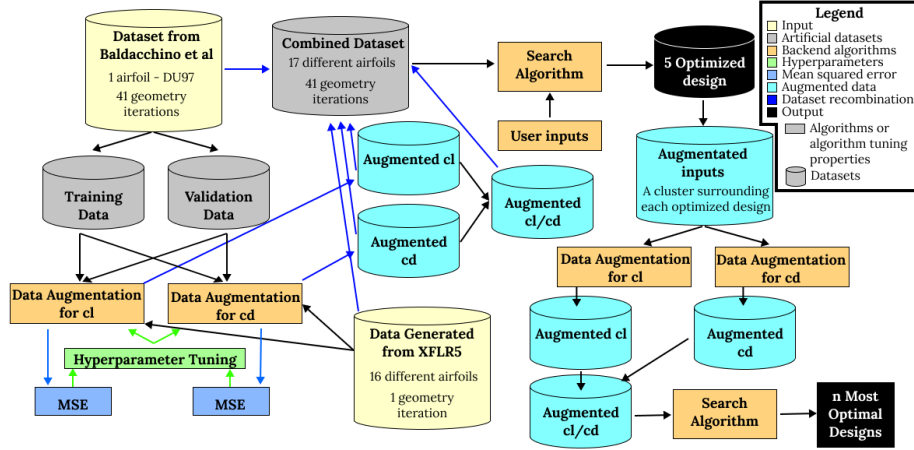


Figure 5: Final model architecture.

An attempt was also made on creating a GAN model with similar functionalities. However, unable to effectively adapt the data to its frameworks, other traditional forms of ML and computer algorithms were chosen over GAN. In addition, to obtain accurate results, the GAN may need to proceed through hundreds of epochs, requiring immense computational power, which contradicts with the prompt of this project. In contrast, although it is heavily influenced by pre-stored data, search algorithms are cost-effective and may obtain high-quality results within seconds. With the aid of the clustering mechanism, the user may receive some of the most optimal results within a few minutes and use the conserved computational power to conduct further experimentations via CFD.

3 Results and Evaluation

3.1 Data Collection and Visualization

We began visualization by creating polar plots with data from Baldacchino et al in hope to gain an understanding in the relationship between various polar properties. Figure 6 illustrates the relationship between cl/cd and alpha of the DU-97 airfoil either with or without VGs installed at various locations. Regardless of the geometric iterations, all plotted relationships between cl/cd and alpha are nonlinear, some even showing degrees of semblance to statistical distributions such as the F-distribution. This corroborates with our hypothesis

that multiple linear regression and transformed linear regressions may not be accurate or feasible in this case.

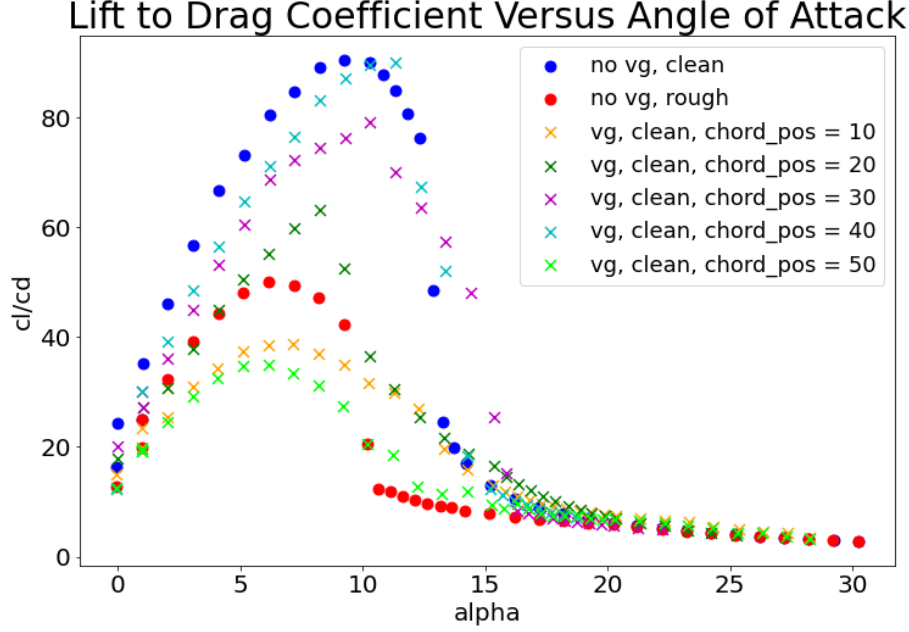


Figure 6: Relationship between cl/cd and α of DU-97 airfoil.

Another interesting finding is although the two most optimal geometrical iteration both have a peak cl/cd of approximately 90, the configuration with VGs reached this peak value at a slightly larger angle of attack, corroborating with the findings of both Forster et al and Baldacchino et al

Figure 7 uses the same geometrical iterations featured in Figure 6, illustrating the relationship between cl/cd . Without VG, regardless of the boundary conditions, after reaching the peak cl value, both airfoils suffered a dramatic drop in cl and gradually restabilized. In contrast, many configurations with VGs can reach their peak cl/cd value, plateau for a few iterations, then suffer the same drop seen in configurations without VGs. Combined with Figure 6, these plots further corroborates with the findings of Baldacchino et al and Forster et al, that VGs may mitigate the dynamic stall on airfoils, allowing them to be installed at a higher angle of attack without encountering a drastic increase in cd .

Figure 8 shows the relationship between cl/cd and alpha of an array of drastically different airfoils. Unlike Figure 6 and 7, these data points are not from an existing dataset; rather, they are simulated data from XFLR5. These simulation data represent airfoils that do not have VGs installed and are tested with a clean boundary layer. A trend seen in this plot is that none of these airfoils reached the cl/cd value achieved by the DU-97 airfoil at similar configurations. We hypothesize that this has occurred because the DU-97 airfoil is a high-lift, low-speed airfoil, while other airfoils featured in this plot are either general purpose or high-speed, low-lift airfoils. Within Figure 8, Gottingen 417a, a modified airfoil reached the highest cl/cd , significantly higher than all other airfoils, which may possibly be a result of its heavy camber. In general,

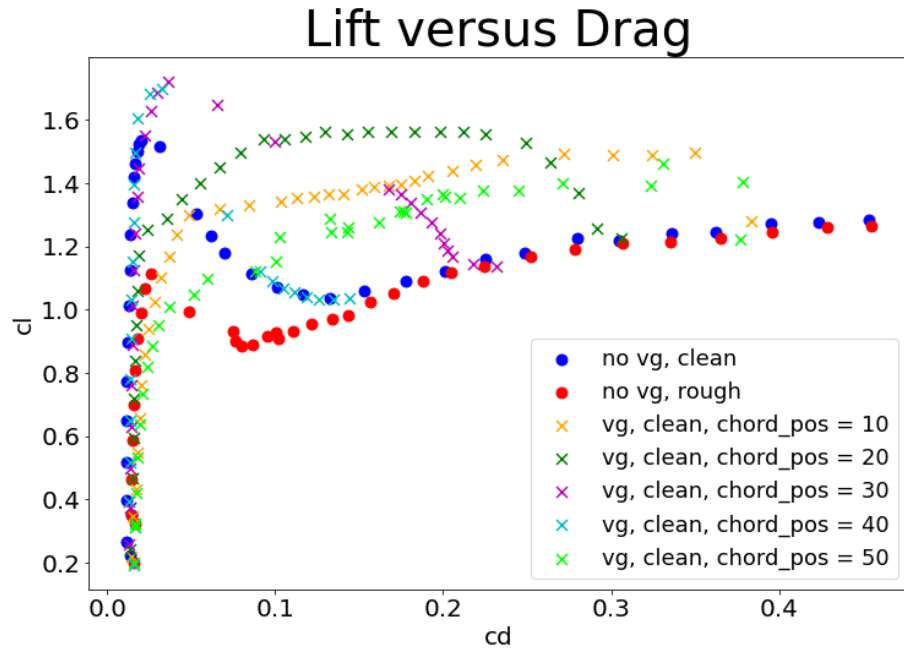


Figure 7: Relationship between cl and cd of DU-97 airfoil.

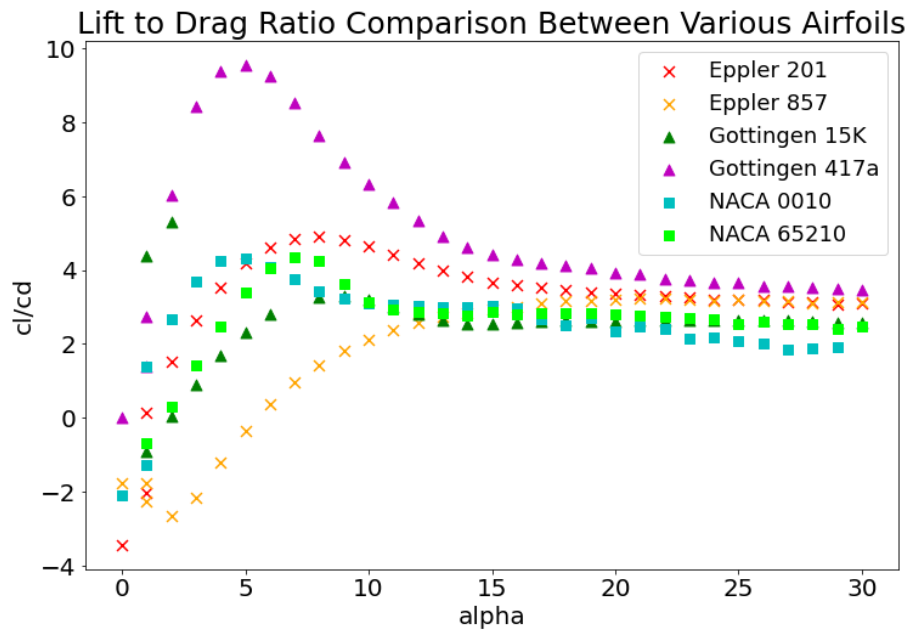


Figure 8: Relationship between cl/cd and α of various airfoils.

the trend exhibited by this plot is heavily cambered or thick airfoils tend to reach a higher peak cl/cd value, while simplistic or high speed airfoils are not as efficient in generating lift.

3.2 Data Augmentation Performance

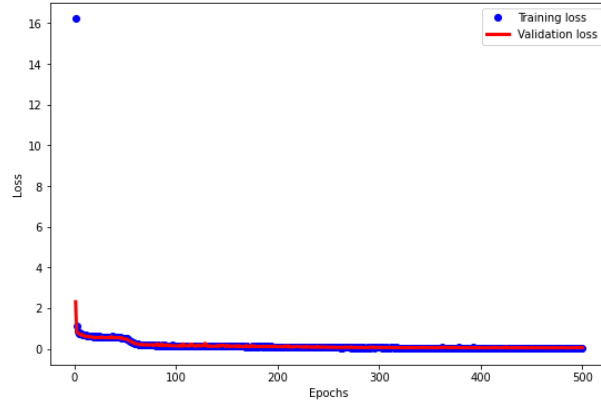


Figure 9: Multilayer perceptron training and validation loss comparison.

Figure 9 portrays the loss values of the multilayer perceptron data augmentation model. After optimizing the model, both the loss value and the validation loss value were able to decrease to approximately 0.04 and the MSE being 0.0348, with some minor fluctuations between each run. However, although the loss and MSE values were at an acceptable level, the model was underfitting and producing unsatisfactory results.

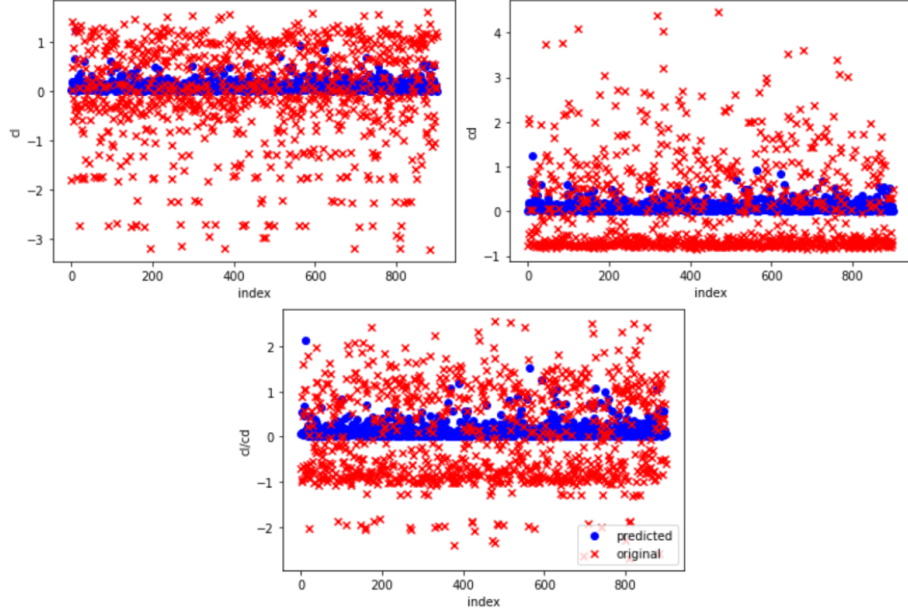


Figure 10: Multilayer perceptron sample accuracy plot.

After extensive testing and tuning, the multilayer perceptron model remains inadequate at accurately predicting the aerodynamic data of airfoils with different geometrical iterations. Regardless of the baseline data, the model continues to produce either low-variance data (Fig. 10) or significantly underfitting data (Fig. 11b). We also noticed that the multilayer perceptron model was predicting data that shows striking semblance to the mirrored image of the original dataset. However, after flipping the results, the prediction was still inaccurate in comparison to most decision tree models.

Figure 11 compares the performance of the five decision tree models and the multilayer perceptron model that we constructed for data augmentation. Other than having a significantly higher MSE value, the multilayer perceptron model plot (Fig. 11b) also graphically demonstrates its inaccuracies. On the other hand, the ExtraTrees (Fig. 11c) and Bagging (Fig. 11d) model outperformed all other models, with their MSE values being significantly smaller in comparison to other contenders. Solely by visualization, they also seem like the best performing models, accurately predicting the validation data. Due to their similar MSE values, we considered their runtimes to assess their computational cost. In the end, ExtraTrees was chosen over Bagging because it achieved optimal accuracy with only 30 `n_estimators`, much less than Bagging, resulting in a much shorter runtime and lower computational cost (Fig. 13).

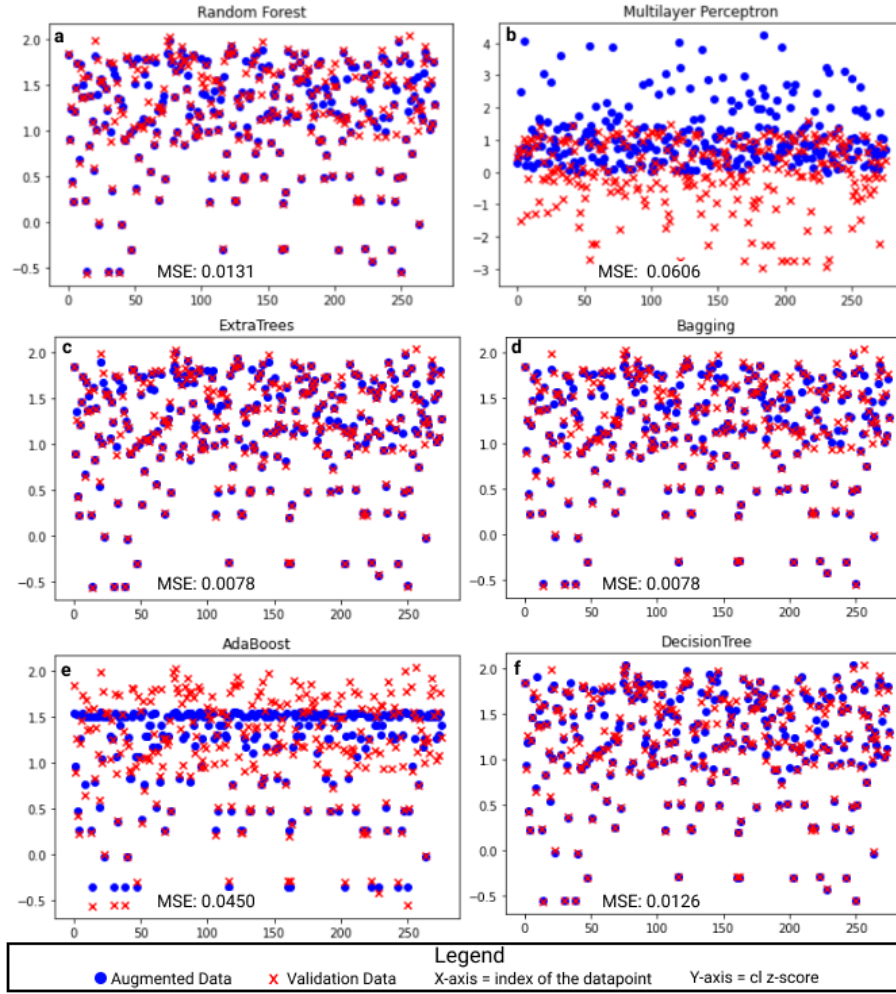


Figure 11: Multilayer perceptron sample accuracy plot.

Figure 12: A sample of hyperparameters of each decision tree model.			
Model Name	n_estimators	min_sample_split	bootstrap
RandomForest	50	2	True
ExtraTrees	30	2	False
Bagging	100	N/A	True
AdaBoost	50	N/A	N/A
DecisionTrees	N/A	2	N/A

Model Name	Runtime (ms)
RandomForest	219
Multilayer Perceptron	4.00e4
ExtraTrees	113
Bagging	386
AdaBoost	72.2
DecisionTrees	48.8

Figure 13: Runtime of models predicting 20287 rows of unseen data.

3.3 Final Model Performance

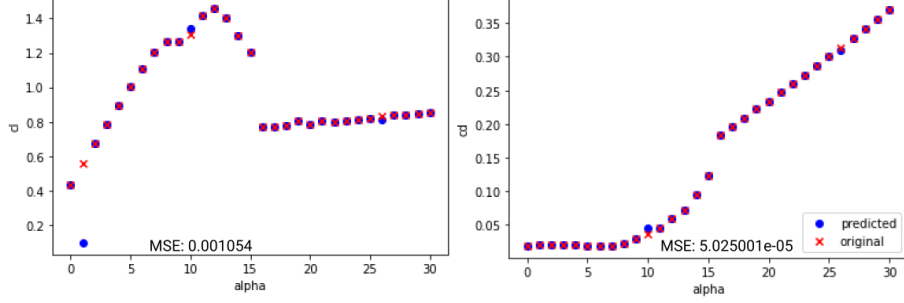


Figure 14: Validation plot with DAE-51 Airfoil.

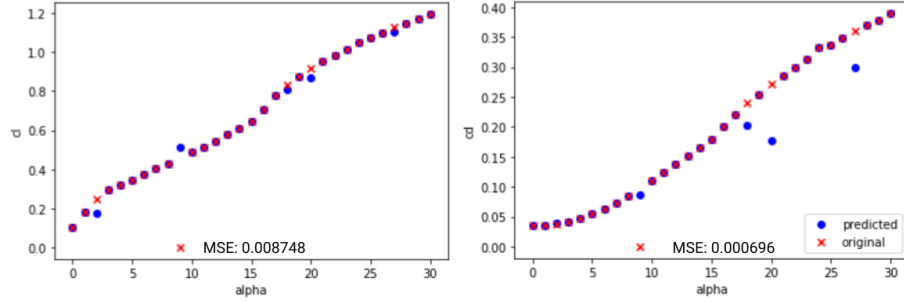


Figure 15: Validation plot with RAF-15 Airfoil.

To validate the results produced by the final prediction model, XFLR5 polar analyses were performed on DAE-51 and RAF-15 airfoils and an attempt on recreating these simulated values using the data augmentation model was made. After simulation and augmentation, their outcomes were exported and plotted for comparison.

The model was able to accurately predict most of the values precisely, although sometimes struggles for consistency. In Figure 14, both for c_l and c_d , with only one or no exception, the predicted value was able to follow the general trend of the c_l and c_d values produced by XFLR5, with a very small MSE value. To further corroborate the performance claim, Figure 15 displays performance of similar caliber, although slightly less consistent. For both plots of Figure 15, the XFLR5 was unable to predict the c_l or c_d value when $\alpha = 9^\circ$. In contrast, the augmentation model was able to make a prediction that roughly followed the general trend, demonstrating that it potentially may be more accurate than CFD if a datapoint becomes unconverged after a large sum of iterations. Both Figure 14 and 15 shows the capability and accuracy of the augmentation model when handling unseen data.

3.4 Observation

Throughout this project, we yielded some interesting observations, especially when optimizing the hyperparameter and when comparing the performance of models.

3.4.1 Optimization of Hyperparameter and Augmentation Performance

A grid search was conducted to seek for the most optimal hyperparameter combinations for each decision tree ML model and a random search was conducted to find the best configurations of the multilayer perceptron model.

After many iterations of optimization on the multilayer perceptron model, it remains incapable of making accurate predictions (Fig. 1, App. D). We hypothesize that this model requires a larger, denser, and more coherent training dataset to extract insight. The original dataset was the merged product of the dataset from Baldacchino et al and XFLR5 simulation results, consisting of 15 features and 3 labels that have very little to no correlations (App. E). Therefore, only with the information of how VGs perform on a single airfoil, it is possible that it cannot accurately predict its behavior because there is not enough data to extract insight.

A potential solution in this case may be increasing hidden layers and neurons to increase the network’s ability to learn the correlation between variables. However, this process did not result in the production of more accurate predictions, only allowing the network to lower its training and validation loss, increasing the accuracy when predicting values near the mean, and remain incapable to predict values that are smaller than the mean or more than a single standard deviation greater than the mean (Fig. 10). This is often categorized as an overfitting issue and it begins to occur as soon as the product of the neurons of all the hidden layers exceeds the number of datapoints in the dataset. In contrast, when the product of neurons in all the hidden layers trails the number of datapoints, the model begins to underfit, creating a strange mirrored image of the original data (Fig. 11). After multiplying all predictions by a factor of -1, the predictions showed some semblance of the original data, although with a much larger loss, MSE, and computational cost in comparison to any decision tree models.

We acknowledge that conducting random search when optimizing hyperparameters may not yield the same result as conducting an effective grid search. However, after conducting hundreds of trials, the current network should be at an optimal configuration. In addition, factoring in the runtime and computational complexity of the multilayer perceptron model, decision tree augmentation models are more superior in this use case because it generates high-quality results at a fraction of the computational cost (Fig. 13).

The grid searches also yielded interesting results. After manually creating a matrix of potential solutions, the most optimal hyperparameters of all decision trees models seem to remain default, with `n_estimator` being the only outlier. Even without any hyperparameter tuning, the models performed significantly better than the multilayer perceptron model with runtimes ranging between 100 to 500 ms. After optimization, the models are simplified and are shallower than the original, yielding even shorter runtimes (Fig. 13).

The trend is potentially caused by the size of the dataset. The default parameters of these models tend to be on the lower end of the acceptable values, allowing great adaptation to smaller datasets and creating a great degree of variability for users with larger datasets. An interesting discovery is that increasing the value of both `min_samples_split` and `min_samples_leaf` decreases the accuracy of the model, changing the `max_depth` and `max_features` increases

fluctuation in predictions (Fig. 2, App. D), and other features have very little impact on the accuracy of data augmentation, though it may impact the total runtime of the program.

We acknowledge that like random searches, the grid searches that we conducted on each augmentation model are relatively small and are inconclusive. However, from the trends produced from these searches, it can be concluded that default hyperparameter values are most likely to produce the most accurate results in this specific use case.

3.4.2 Computational cost comparison between ML models, CFD, and wind tunnel testing

As shown in section 3.3, this model can accurately predict the polar performance of an airfoil without the installation of VG or complicated boundary conditions. Unfortunately, accuracy assessments on unseen data augmentation with VG beyond validation data cannot be conducted without access to CFD.

Model Name	Runtime (ms)
RandomForest	1.08e-2
Multilayer Perceptron	1.97
ExtraTrees	5.57e-3
Bagging	1.90e-2
AdaBoost	3.56e-3
DecisionTrees	2.41e-3

Figure 16: Average runtime of each model for predicting a single datapoint.

However, a potential assessment on its performance is the time and cost of executing experiments using these models. As you can see (Fig. 16), any of these ML models, including the multilayer perceptron, may simulate thousands of geometrical iterations within a single second, minimizing the computational cost of prototype testing. We acknowledge that the accuracy and flexibility of wind tunnel testing and CFD are definitely superior. However, the runtime improvements in using ML regressors are also significant and unignorable.

In comparison to 2D simulation software that utilizes either the Kutta-Joukowski theorem or the lifting-line theory, at similar accuracy caliber, this machine learning approach is faster when generating batches of data, but slower when conducting individual analysis, because singular analysis cannot utilize the clustering mechanism to generate geometries and the model must train itself every time before making a prediction. However, the final model has three major advantages against 2D airfoil simulation software.

First, the final model is not purely a CFD, but in consort with a generative design algorithm. Therefore, its intent is not conducting analysis on airfoils; rather, it focuses on providing the user with some of the most optimized designs to reduce their candidates for simulation, thus decreasing the total simulation time and computational cost.

Another advantage of this program is that it also includes the factor of VG, which is a 3D feature. Most airfoil simulators are only capable of conducting 2D

simulations, without any variability for airfoil accessories. Therefore, this model offers more flexibility and a broader scope than traditional airfoil simulators.

In addition, this program will not encounter unconvergence issues. Although it lacks accuracy when predicting for unconverged datapoints (Fig. 15), it can follow the general trend of most of its predictions, rather than resulting in large fluctuations in data.

However, in comparison to CFD and wind tunnel testing, the only advantage that this algorithm yields is the simulation time. Due to the lack of accessibility, we are unable to quantitatively compare the runtime and computational of simulating similar geometries with other methods. However, when qualitatively comparing the method of topologically optimizing airfoils and VG configurations using various techniques, it is clear that this model is significantly faster and cheaper. Unlike CFD and wind tunnel testing, this model is intuitive, requires no initial setup or analysis definition, and may be modified from the source code to adjust usability. Complicated geometry preparations that CFD relies on are not necessary for this program, thus increasing the usability and reducing the time required to conduct simulations. Unlike most 3D CFD and wind tunnel testing, this program can also conduct batch analysis to produce optimized designs, increasing the efficiency of optimization.

The model’s main advantage, usability and simplicity, is also its major pitfall. Although CFD and wind tunnel testing require extensive geometry preparation or fabrication, external condition definition, and longer solve time, these inputs will all be simulated, producing outputs with higher dimensionality at an astonishing accuracy, unmatched by this algorithm. Therefore, we acknowledge that the accuracy and efficacy of this program cannot be compared with traditional testing methods.

3.5 Significance and Broader Context

Overall, this project yields a functional generative design model that may generate and evaluate polar performance of an airfoil and VG combination to find the most optimal design option. Depending on the breadth of the constraints, it was able to output optimal designs between 50 to 80 seconds of runtime, including artificially creating clusters of inputs and outputs. This is significant because it is exponentially faster than any traditional method and many generative design models. Although this model may not be as accurate, it shows the potential of ML applications in the field and numerically assessed the cost reduction impact of such applications. It also warrants further research to compare the performance of various ML techniques in this field, create a stepping stone for additional development on the current model and explorations for other more radical approaches.

4 Conclusion

In this project, we created a model that functions as a hybrid of CFD simulation and generative design programs in hope to reduce the computational cost of topology optimization. After tuning the hyperparameters, the final data augmentation model may conduct batch analysis on thousands of rows of data at the average speed of $5.57e-3$ ms per row, with a mean squared error (MSE)

of 0.0078. After expanding the baseline dataset through augmentation, it also gained the ability to accurately predict unseen data, with MSE values ranging between 0.01 to 5.00e-5. Depending on the breadth of the constraints, the final model was able to output n optimal designs within 50-80 seconds, significantly faster than CFD or other generative design models. Although we are unable to assess if the output from the final model is truly the most optimal under the constraints, other accuracy assessments suggest that this is most likely to be the case, although adding additional baseline data and further optimizing the hyperparameters may yield superior results.

5 Limitation and Future Work

We acknowledge there are many limitations to this program. Despite extensive hyperparameter optimization, the initial baseline dataset was insufficient for making predictions at a similar accuracy caliber of CFD. This also ties together with the lack of resources, mainly model diversity in datasets, hindering the accuracy and validity of the data augmentation model. Our lack of access to CFD also disallows us from further verifying the prediction accuracy with rough boundary conditions and VG installations, resulting in a lack of quantitative affirmation of accuracy and credibility.

In the future, further studies on accuracy improvements of the current model, source code organization, and implementation of other machine learning techniques may be extremely helpful in increasing the prevalence of similar models.

To increase accuracy, it may be best to use CFD to establish the accuracy of the data augmentation model when predicting unseen data with VGs or if it warrants further tuning. In addition, adding other existing datasets to the model may increase the scope and accuracy of the augmentation model. Also, using other augmentation models to fill in the sparse spaces within the current dataset may allow it to make better predictions when working with dense, unseen data.

Although the computational cost of this program is already quite low, the efficiency of this program will benefit from using higher level libraries and further organization of code. This also ties into further studies on similar topics using techniques such as GAN, dimensionality reduction (DR), etc. A potential direction for further research is to compare the performance of the current algorithm with other models that implement distinct ML techniques, to find the most accurate yet computationally cost-effective program.

6 Reference

- [1] T. Norton and D.-W. Sun, “Computational fluid dynamics (CFD) – an effective and efficient design and analysis tool for the food industry: A review,” *Trends in Food Science & Technology*, vol. 17, no. 11, pp. 600–620, Nov. 2006, doi: 10.1016/j.tifs.2006.05.004.
- [2] J. Jeon and S.J. Kim, “FVM Network to Reduce Computational Cost of CFD Simulation,” *ArXiv*, arXiv:2105.03332v1.
- [3] W. Chen, K. Chiu, and M. Fuge, “Aerodynamic Design Optimization and Shape Exploration using Generative Adversarial Networks,” *AIAA Scitech 2019 Forum*, Jan. 2019, doi: 10.2514/6.2019-2351.
- [4] T. Hauser, T. I. Mattox, R. P. LeBeau, H. G. Dietz, and P. G. Huang, “High-Cost CFD on a Low-Cost Cluster,” *ACM/IEEE SC 2000 Conference (SC’00)*, 2000, doi: 10.1109/sc.2000.10020.
- [5] S. Sinha, V. Sundeeep, and Ravande, “Drag Reduction of Natural Laminar Flow Airfoils with a Flexible Surface Deturbulator,” *AIAA Flow Control Conference*, vol. 5, no. 8, 2006, Accessed: Jul. 21, 2021. [Online]. Available: <http://www.deturbulator.com/AIAA-2006-3030-245.pdf>.
- [6] D. Baldacchino, C. Ferreira, D. D. Tavernier, W. A. Timmer, and G. J. W. van Bussel, “Experimental parameter study for passive vortex generators on a 30% thick airfoil,” *Wind Energy*, vol. 21, no. 9, pp. 745–765, Apr. 2018, doi: 10.1002/we.2191.
- [7] D. De Tavernier, C. Ferreira, A. Viré, B. LeBlanc, and S. Bernardy, “Controlling dynamic stall using vortex generators on a wind turbine airfoil,” *Renewable Energy*, vol. 172, pp. 1194–1211, Jul. 2021, doi: 10.1016/j.renene.2021.03.019.
- [8] K. J. Forster and T. R. White, “Numerical Investigation into Vortex Generators on Heavily Cambered Wings,” *AIAA Journal*, vol. 52, no. 5, pp. 1059–1071, May 2014, doi: 10.2514/1.j052529.
- [9] Y. Zhao, H. D. Akolekar, J. Weatheritt, V. Michelassi, and R. D. Sandberg, “RANS turbulence model development using CFD-driven machine learning,” *Journal of Computational Physics*, vol. 411, p. 109413, Jun. 2020, doi: 10.1016/j.jcp.2020.109413.
- [10] E. Gunpinar, U. C. Coskun, M. Ozsipahi, and S. Gunpinar, “A Generative Design and Drag Coefficient Prediction System for Sedan Car Side Silhouettes based on Computational Fluid Dynamics,” *Computer-Aided Design*, vol. 111, pp. 65–79, Jun. 2019, doi: 10.1016/j.cad.2019.02.003.
- [11] S. Bhatnagar, Y. Afshar, S. Pan, K. Duraisamy, and S. Kaushik, “Prediction of aerodynamic flow fields using convolutional neural networks,” *Computational Mechanics*, vol. 64, no. 2, pp. 525–545, Jun. 2019, doi: 10.1007/s00466-019-01740-0.
- [12] T. von Kármán and H. Tsien, “Lifting-Line Theory for a Wing in Non-uniform Flow,” *Collected Works of H.S. Tsien (1938–1956)*, pp. 286–296, 2012, doi: 10.1016/b978-0-12-398277-3.50017-8.

7 Appendices

7.1 Appendix A - Summary of the Original Dataset

	vg	boundary	vg_chord_pos	vg_h	vg_l	vg_d_int	vg_d_ext	vg_beta
count	21463	21463	21463	21463	21463	21463	21463	21463
mean	0.9296	0.6817	24.9341	5.2561	15.6452	18.3965	37.1564	14.1904
std	0.2557	0.4658	13.2165	2.8991	8.6001	10.1469	20.3925	4.3891
min	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
25%	1.0000	0.0000	20.0000	2.5000	7.5000	8.7500	17.5000	15.0000
50%	1.0000	1.0000	20.0000	5.0000	15.0000	17.5000	35.0000	15.0000
75%	1.0000	1.0000	30.0000	5.0000	15.0000	17.5000	35.0000	15.0000
max	1.0000	1.0000	50.0000	10.0000	30.0000	35.0000	70.0000	25.0000

	camber	camber_pos	thickness	thicknes_pos	alpha	cl	cd	cl/cd
count	21463	21463	21463	21463	21463	21463	21463	21463
mean	4.0464	44.1470	15.0364	33.1796	14.8355	0.9928	0.1145	15.8073
std	2.5079	18.6216	10.0778	11.3258	8.8978	0.4142	0.1002	13.0946
min	0.0000	0.0000	3.2000	2.2000	-6.2500	-0.5687	0.0110	-36.0912
25%	2.1000	40.4000	8.6600	29.6300	7.0000	0.7133	0.0294	5.7198
50%	4.0000	48.0600	11.8900	31.6300	15.0000	1.0946	0.0710	10.8582
75%	5.7300	52.3500	18.9800	39.7400	22.0000	1.3016	0.1926	23.4399
max	9.1900	81.0800	45.0000	49.9500	30.2500	2.0753	0.4964	90.5103

Figure 1: Summary of the Original Dataset

7.2 Appendix B - Summary of the Augmented Dataset

	vg	boundary	vg_chord_pos	vg_h	vg_l	vg_d_int	vg_d_ext	vg_beta
count	1623	1623	1623	1623	1623	1623	1623	1623
mean	0.6808	0.7640	18.0160	3.8031	11.3078	13.3110	26.8438	10.4054
std	0.4663	0.4247	15.6293	3.3561	9.9503	11.7462	23.6409	7.3126
min	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.2500	0.0000	1.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.5000	1.0000	1.0000	20.0000	5.0000	15.0000	17.5000	35.0000	15.0000
0.7500	1.0000	1.0000	30.0000	5.0000	15.0000	17.5000	35.0000	15.0000
max	1.0000	1.0000	50.0000	10.0000	30.0000	35.0000	70.0000	25.0000

	camber	camber_pos	thickness	thicknes_pos	alpha	cl	cd	cl/cd
count	1623	1623	1623	1623	1623	1623	1623	1623
mean	2.6201	70.1635	25.7806	30.8289	12.8248	1.0358	0.1144	21.7807
std	1.5894	19.9076	8.5559	6.5360	8.0403	0.5762	0.1077	23.6078
min	0.0000	0.0000	3.2000	2.2000	-6.2500	-0.5687	0.0110	-36.0912
0.2500	2.1000	55.3600	24.0200	29.6300	6.1850	0.6477	0.0232	3.9183
0.5000	2.1000	81.0800	30.0000	29.6300	13.3900	1.0913	0.0797	10.7478
0.7500	2.1000	81.0800	30.0000	29.6300	18.4350	1.5053	0.1700	43.2069
max	9.1900	81.0800	45.0000	49.9500	30.2500	2.0753	0.4964	90.5103

Figure 1: Summary of the Augmented Dataset

7.3 Appendix C - Summary of Airfoils Featured in this Project

The following airfoils are featured in this project:

DAE-51, DU-97, Eppler-49, Eppler-58, Eppler-201, Eppler-1098, Eppler-678, Eppler-857, Gottingen-12k, Gottingen-15k, Gottingen-118, Gottingen-234, Gottingen-417a, NACA-0010, NACA-16006, NACA-2415, NACA-4424, NACA-65210, RAF-15

Please refer to the following repository to access the airfoil coordinates:
https://github.com/GraceYuMeadowridge/SSI_Project_Airfoils_Database.git

7.4 Appendix D - Problematic Prediction Samples

index	camber	camber_pos	thickness	thicknes_pos	alpha	cl	cd	cl/cd
19498	2.5	81.48	30.4	29.63	8.33	1.256507	0.013816	90.9428
9994	2.3	81.38	30.4	30.03	8.33	1.256507	0.013816	90.9428
18943	2.5	81.18	30.1	29.73	8.33	1.256507	0.013816	90.9428
10051	2.3	81.48	30.1	29.93	8.33	1.256507	0.013816	90.9428
654	2.1	81.48	30	30.03	8.33	1.256507	0.013816	90.9428
18937	2.5	81.18	30.1	29.63	8.33	1.256507	0.013816	90.9428
18930	2.5	81.18	30	30.03	8.33	1.256507	0.013816	90.9428
661	2.1	81.48	30.1	29.63	8.33	1.256507	0.013816	90.9428
10045	2.3	81.48	30.1	29.83	8.33	1.256507	0.013816	90.9428
18924	2.5	81.18	30	29.93	8.33	1.256507	0.013816	90.9428
667	2.1	81.48	30.1	29.73	8.33	1.256507	0.013816	90.9428
18918	2.5	81.18	30	29.83	8.33	1.256507	0.013816	90.9428
10039	2.3	81.48	30.1	29.73	8.33	1.256507	0.013816	90.9428
18912	2.5	81.18	30	29.73	8.33	1.256507	0.013816	90.9428
673	2.1	81.48	30.1	29.83	8.33	1.256507	0.013816	90.9428
18906	2.5	81.18	30	29.63	8.33	1.256507	0.013816	90.9428
10033	2.3	81.48	30.1	29.63	8.33	1.256507	0.013816	90.9428
679	2.1	81.48	30.1	29.93	8.33	1.256507	0.013816	90.9428
10026	2.3	81.48	30	30.03	8.33	1.256507	0.013816	90.9428
685	2.1	81.48	30.1	30.03	8.33	1.256507	0.013816	90.9428
10020	2.3	81.48	30	29.93	8.33	1.256507	0.013816	90.9428
10014	2.3	81.48	30	29.83	8.33	1.256507	0.013816	90.9428
692	2.1	81.48	30.2	29.63	8.33	1.256507	0.013816	90.9428
10008	2.3	81.48	30	29.73	8.33	1.256507	0.013816	90.9428
10002	2.3	81.48	30	29.63	8.33	1.256507	0.013816	90.9428
18949	2.5	81.18	30.1	29.83	8.33	1.256507	0.013816	90.9428
648	2.1	81.48	30	29.93	8.33	1.256507	0.013816	90.9428
18955	2.5	81.18	30.1	29.93	8.33	1.256507	0.013816	90.9428
622	2.1	81.38	30.4	30.03	8.33	1.256507	0.013816	90.9428
19023	2.5	81.18	30.3	30.03	8.33	1.256507	0.013816	90.9428

Figure 1: Demonstration of identical predictions from unoptimized augmentation models.

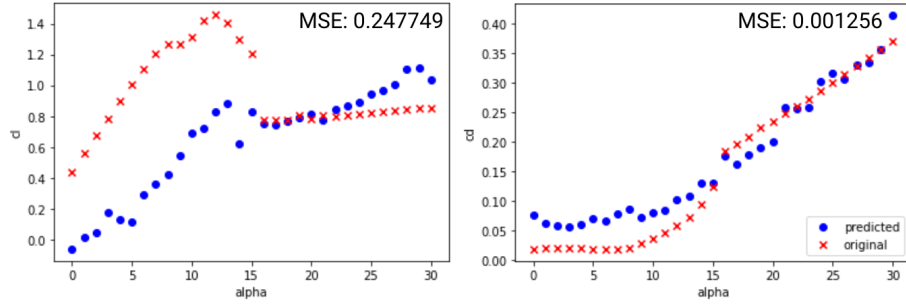


Figure 2: Demonstration of fluctuation in predictions of unoptimized augmentation models.

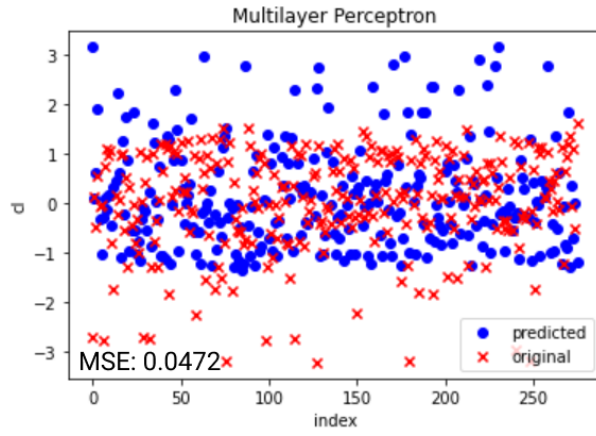


Figure 3: Adapted from Fig. 11b, comparison between original data and flipped prediction data.

7.5 Appendix E - Correlation Matrix of Relationships Between Variables

	vg	boundary	vg_chord_pos	vg_h	vg_l	vg_d_int	vg_d_ext	vg_beta
vg	1.0000	-0.2716	0.7895	0.7761	0.7783	0.7761	0.7777	0.9746
boundary_l	-0.2716	1.0000	-0.3334	-0.2253	-0.2336	-0.2253	-0.2187	-0.2451
vg_chord_pos	0.7895	-0.3334	1.0000	0.7513	0.7589	0.7513	0.7478	0.7585
vg_h	0.7761	-0.2253	0.7513	1.0000	0.9694	1.0000	0.9971	0.7518
vg_l	0.7783	-0.2336	0.7589	0.9694	1.0000	0.9694	0.9668	0.7542
vg_d_int	0.7761	-0.2253	0.7513	1.0000	0.9694	1.0000	0.9971	0.7518
vg_d_ext	0.7777	-0.2187	0.7478	0.9971	0.9668	0.9971	1.0000	0.7529
vg_beta	0.9746	-0.2451	0.7585	0.7518	0.7542	0.7518	0.7529	1.0000
camber	-0.4781	0.1819	-0.3775	-0.3711	-0.3721	-0.3711	-0.3718	-0.4659
camber_pos	0.8011	-0.3048	0.6325	0.6218	0.6235	0.6218	0.6230	0.7808
thickness	0.7205	-0.2742	0.5688	0.5592	0.5608	0.5592	0.5603	0.7022
thicknes_pos	-0.2680	0.1020	-0.2116	-0.2080	-0.2086	-0.2080	-0.2084	-0.2612
alpha	-0.1534	0.0186	-0.1519	-0.1594	-0.1605	-0.1594	-0.1605	-0.1453
cl	0.4322	-0.0634	0.2725	0.3321	0.3302	0.3321	0.3355	0.4373
cd	-0.4863	0.0899	-0.4053	-0.4239	-0.4251	-0.4239	-0.4266	-0.4769
cl/cd	0.4504	-0.0412	0.3765	0.3627	0.3673	0.3627	0.3659	0.4449

	camber	camber_pos	thickness	thicknes_pos	alpha	cl	cd	cl/cd
vg	-0.4781	0.8011	0.7205	-0.2680	-0.1534	0.4322	-0.4863	0.4504
boundary_l	0.1819	-0.3048	-0.2742	0.1020	0.0186	-0.0634	0.0899	-0.0412
vg_chord_pos	-0.3775	0.6325	0.5688	-0.2116	-0.1519	0.2725	-0.4053	0.3765
vg_h	-0.3711	0.6218	0.5592	-0.2080	-0.1594	0.3321	-0.4239	0.3627
vg_l	-0.3721	0.6235	0.5608	-0.2086	-0.1605	0.3302	-0.4251	0.3673
vg_d_int	-0.3711	0.6218	0.5592	-0.2080	-0.1594	0.3321	-0.4239	0.3627
vg_d_ext	-0.3718	0.6230	0.5603	-0.2084	-0.1605	0.3355	-0.4266	0.3659
vg_beta	-0.4659	0.7808	0.7022	-0.2612	-0.1453	0.4373	-0.4769	0.4449
camber	1.0000	-0.2363	-0.4940	-0.1643	0.0563	-0.1395	0.2611	-0.2470
camber_pos	-0.2363	1.0000	0.7594	-0.2635	-0.1208	0.4140	-0.4454	0.4339
thickness	-0.4940	0.7594	1.0000	-0.2292	-0.0977	0.3101	-0.3734	0.3785
thicknes_pos	-0.1643	-0.2635	-0.2292	1.0000	0.0624	-0.1888	0.1412	-0.1542
alpha	0.0563	-0.1208	-0.0977	0.0624	1.0000	0.5987	0.8198	-0.2067
cl	-0.1395	0.4140	0.3101	-0.1888	0.5987	1.0000	0.0928	0.4812
cd	0.2611	-0.4454	-0.3734	0.1412	0.8198	0.0928	1.0000	-0.5734
cl/cd	-0.2470	0.4339	0.3785	-0.1542	-0.2067	0.4812	-0.5734	1.0000

Figure 1: Correlation Matrix of Relationships Between Variables.